

Database Assignment 03.

Name: Yash Raj

Roll no: 21k-0737

Question #1.

part(i):-

$\pi_{platform_Name}(PLATFORM \bowtie ENROLLMENT \bowtie$
 $\sigma_{Difficulty_level = 'Advanced'})$

Exp:- Select courses with difficulty level as "Advanced"

part(ii):-

$\pi_{instructor_Name}(INSTRUCTOR \bowtie COURSE_ACTIVITY$
 $\bowtie \sigma_{Category = 'DataScience'})$

Exp:- find instructors associated with Data-science & status 'Completed'

part(iii):-

$\pi_{platform_Name}(PLATFORM) - \pi_{platform_Name}(PLATFORM$
 $\bowtie INSTRUCTOR)$

Exp:- Subtract platform having instructor from all platforms.

part(iv):-

$\pi_{Course_ID, Course_Name}(COURSE \bowtie ENROLLMENT)$

Exp:- Join course with enrollment records to identify

part(v):-

$\pi_{Course_Name, platform_Name, completion_Rate}$
 $(\sigma_{Category = 'AI' \wedge completion_Rate > 85}(COURSE$
 $\bowtie ENROLLMENT))$

Exp:- Display AI courses along with platform names where completion rate > 85

Name: Yash Raj

Roll no: 24K-0737

Question #2:

part(i):-

Relational Algebra:-

$\pi_{title}(\sigma_{isAvailable = 0 \wedge price > 1000}(BOOK))$

Meaning:-

Display title of Unavailable books whose price is greater than 1000.

Query:-

Select title

from Book

Where isAvailable = 0 AND price > 1000;

Ans

part(ii):-

Relational Algebra:-

$\pi_{title}(\sigma_{(genre = 'fiction' \wedge price > 800) \vee genre = 'History'}(BOOK))$

Meaning:- Show title of fiction books, where price is more than 800 or books belonging to history genre.

Query:-

Select title

from Book

Where (genre = "fiction" AND price > 800) OR genre = "History";

Ans

Name: Yash Raj

Roll no: 2410-07

part (iii)

Meaning

Display title & price of available
Science - books.

Query

Select title, price
from Book
Where genre = 'Science' AND
isAvailable = 1;

My

part (iv)

Meaning: Show title & one-tenth of
the price of available Science
books.

Query

Select title, price/10
from Book
Where genre = 'Science' & isAvailable = 1;

My

Names / Ash Raj

Roll no: 24K-0737

part(v).

Meaning: Display publisher ID that published the book 'Al-Banai'.

Query:

Select publisherID
from publisher P
Join Book B on P.bookID = B.bookID
Where B.title = 'Al-Banai';

Mr.

part(vi) -

Meaning: Display staff names & library names where the staff address matches the library city.

Query:

Select staff.name, library.libName
from Staff
Join library on Staff.libID = library.libID
Where Staff.address = library.city;

Mr.

Name: Yash Raj

Roll-no: 24K0737

part (vii)

Meaning: Display IDs of staff members
when salary is not equal to
60000.

Query:

Select staffID
from staff
where salary <> 60000; *Ans*

part (viii)

Meaning: Display publisher ID existing in both
Karachi & Islamabad.

Query:

Select publisherID
from publisher
where city = "Karachi"
INTERSECT

Select publisherID
from publisher
where city = "Islamabad".

Name: Yash Raj

Roll no: 24K0737

part (ix):-

Meaning:- Display title of books that have not been published.

Query:-

select title

from Book

where bookID not in (select bookID
from publisher);

Question #3:-

Transaction:- T₁: read-item(A)

A: A-N

write-item(A)

T₂: read-item(A)

A: A+M

write-item(A)

part (i):- Non-serial schedule:-

r₁(A)

r₂(A)

w₁(A)

w₂(A)

Name: Yash Raj

Roll no: 24K-0737

part(ii): Precedence Graph:

Conflict:

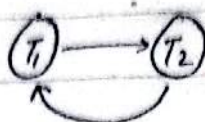
(i) $r_1(A)$ before $w_2(A) \rightarrow$ Edge $T_1 \rightarrow T_2$

(ii) $r_2(A)$ before $w_1(A) \rightarrow$ Edge $T_2 \rightarrow T_1$

Graph:

$T_1 \rightarrow T_2$

$T_2 \rightarrow T_1$



part(iii):

The schedule is not conflict-serializable because the precedence contains a cycle.

part(iv):

Execute T_1 fully before T_2 or T_2 fully before T_1 .

Ex:

T_1 then T_2 :

read(A); $A := A - N$; write(A) $\rightarrow T_1$ completes

read(A); $A := A + M$; write(A) $\rightarrow T_2$ completes

part(v): Real world impact:

This causes inconsistent account balances & lost updates in banking systems.

Name: Yash Raj

Roll no: 24K0737

Question #4:

1) Schedule S1:

$r_1(x), r_2(y), r_3(x), r_4(y), w_1(x),$
 $w_2(y), w_3(x), w_4(y)$

Conflicts on x:

$r_1(x)$ before $w_3(x) \rightarrow T_1 \rightarrow T_3$

$r_3(x)$ before $w_1(x) \rightarrow T_3 \rightarrow T_1$

Conflicts on y:

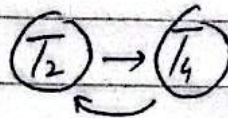
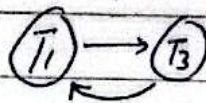
$r_2(y)$ before $w_4(y) \rightarrow T_2 \rightarrow T_4$

$r_4(y)$ before $w_2(y) \rightarrow T_4 \rightarrow T_2$

Graph:

$T_1 \leftrightarrow T_3$

$T_2 \leftrightarrow T_4$



$\Rightarrow$ S1 is not serializable
Because cycle exist in the graph.

2) Schedule S2: $r_3(x), w_3(x), r_1(x), r_1(y), w_1(x),$
 $r_2(y), w_2(y), r_4(y), w_4(y)$

Conflicts on x:

$w_3(x)$ before $r_1(x) \rightarrow T_3 \rightarrow T_1$

$w_3(x)$ before $w_1(x) \rightarrow T_3 \rightarrow T_1$

Conflict on y:

$w_2(y)$ before $r_4(y) \rightarrow T_2 \rightarrow T_4$

$w_2(y)$ before $w_4(y) \rightarrow T_2 \rightarrow T_4$

Name: Yath-Ray

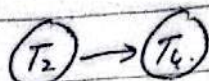
Roll no: 21K-073

Graph:-

$T_3 \rightarrow T_1$



$T_2 \rightarrow T_4$



→ Yes it is serializable. Because the graph has no cycles.

2) Equivalent Serial Order:-

$T_3 \rightarrow T_1 \rightarrow T_2 \rightarrow T_4$

Ans

Question #1:-

part(i):- This problem is "lost update problem" because one transaction overwrites the update of another transaction.

part(ii):- final-Balance after Execution:-

Execution:-

T_1 reads 5000

T_2 reads 5000

T_1 computes 4000

T_2 computes 7000

T_1 writes 4000

T_2 writes 7000

final-Balance is 7000

Ans

Name: Yash Raj

Roll no: 24K-0737

part(iii): Correct Serial-Balance:

$$5000 - 1000 + 2000 = 6000$$

Correct-Balance: 6000 pm

↑
violation of Serial-Order.

ms

Question #6:

part(a):

Timestamp ordering protocol:-

- Each transaction receives a unique timestamp when it starts.
- Older transactions must execute before newer transactions.

Ex:- $TS(T_1) = 1$
 $TS(T_2) = 2$

2) problems:-

(i) Transaction-Restart:-

A younger transaction may be rolled back repeatedly.

(ii) Starvation:- A transaction may never complete because it continuously restarts.

(iii) Cascading Roll-back:-

If one transaction fails dependent transaction may also fail.

Name: Yash Raj

Roll no: 24K-0737

1) Strict timestamp ordering

Strict timestamp ordering delays writes until the transaction commits. It prevents dirty reads & cascading rollback.

Ex:-

T₁ writes A

T₂ ~~writes~~ to read A before T₁ commits
tries

→ In strict timestamp ordering:-

T₂ wait until T₁ commits. (Thus consistency is maintained)

Question #6:-

part (b) :-

Transactions:-

T₁ transfers 300 from A to B

T₂ transfers 200 from B to A

2) Deadlock Situation:-

Ex:- T₁ lock Account A

T₂ lock Account B

T₁ wait for B

T₂ wait for A

Both wait forever & this situation is called 'Deadlock'.

Name: Yash Raj.

Roll. no: 24K0737

i) Deadlock prevention:

Resource Ordering: Always lock accounts in the same order.

Ex.: lock A first, then B

→ This prevents circular waiting.

ii) Deadlock Resolution:

Timeout/Deadlock Detection:

The DBMS detects cycle & abort one transaction.

Ex.:

Abort T₂

Release locks

Allow T₁ to continue

→ This resolves the deadlock.

AS